DOCUMENTAÇÃO · VERSÃO 1.2.0

Cesto

Banco de dados

Os seis bancos PostgreSQL do projeto: diagramas, tabelas, índices, máquinas de estado, outbox, chaves do Redis e como preparar o ambiente.

Copyright (©) 2026 Clara A. Portes
Licença MIT

Versão 1.2.0
Outubro de 2026

Sumário

1 Visão geral 4
Um banco por serviço 4
Como os dados se relacionam entre serviços 4
Redis 4
Convenções 5

2 cesto_auth 6
Diagrama 6
users 6
oauth_clients 7
authorization_codes 7
refresh_tokens 7
signing_keys 8

3 cesto_catalog 9
Diagrama 9
categories 9
products 9
outbox_events 10

4 cesto_inventory 11
Diagrama 11
stock_items 11
reservations 11
processed_events e outbox_events 12

5 cesto_orders 13
Diagrama 13
orders 13
order_items 14
order_history 14

6 cesto_payments e cesto_notifications 15
Diagrama 15
payments 15
notifications 16

Copyright (©) 2026 Clara A. Portes · Licença MIT

7 Máquinas de estado 17
Pedido 17
Reserva de estoque 17
Refresh token 18
Pagamento 18

8 Eventos, outbox e Redis 19
Outbox transacional 19
Chaves do Redis 19
Envelope de evento 19

9 Como preparar o banco 20
Com Docker (recomendado) 20
PostgreSQL próprio 20
Mudar o esquema 20
Backup e retenção 21
Consultas úteis 21

Copyright (©) 2026 Clara A. Portes · Licença MIT

1 Visão geral

Como os dados estão divididos entre os serviços e as convenções usadas em todos os bancos.

Um banco por serviço

O Cesto segue o padrão *database per service*: cada microsserviço é o único dono do seu banco. Assim cada parte pode evoluir, escalar e ter backup separadamente, e uma mudança de tabela não quebra outro serviço.

BANCO	DONO	CONTEÚDO
`cesto_auth`	`auth`	Usuários, clientes OAuth2, códigos, refresh tokens, chaves de assinatura
`cesto_catalog`	`catalog`	Categorias, produtos, outbox
`cesto_inventory`	`inventory`	Saldos, reservas, outbox, eventos processados
`cesto_orders`	`orders`	Pedidos, itens, histórico, outbox, eventos processados
`cesto_payments`	`payments`	Pagamentos, outbox, eventos processados
`cesto_notifications`	`notifications`	Notificações

Os seis bancos podem ficar no mesmo servidor PostgreSQL 16 (como no Docker Compose) ou em servidores diferentes. Basta apontar o `DATABASE_URL` de cada serviço. Cada serviço abre a conexão em `src/db.js`, com o adapter `@prisma/adapter-pg` e o cliente Prisma gerado por `npm run generate` em `src/generated/prisma` (fora do Git). Esse cliente é gerado em TypeScript e o Node.js 22.18 ou mais novo o carrega direto, removendo os tipos; o resto do código é JavaScript.

Como os dados se relacionam entre serviços

Não existem chaves estrangeiras entre bancos. As referências cruzadas são identificadores (`product_id`, `order_id`, `user_id`) mantidos por eventos:

- `products.id` do catálogo vira `stock_items.product_id` no estoque pelo evento `product.created`.
- `orders.id` é referenciado por `reservations.order_id`, `payments.order_id` e `notifications.order_id`.
- `users.id` do auth é o `sub` do JWT e aparece como `user_id` em pedidos, pagamentos e notificações.
- Os itens do pedido guardam uma cópia do SKU, do nome e do preço no momento da compra.

Redis

O Redis não é fonte da verdade de nenhum dado de negócio, exceto o carrinho, que é descartável por natureza. Ele guarda os fluxos de eventos, o carrinho e os contadores de rate limit.

Copyright (©) 2026 Clara A. Portes · Licença MIT

> **FORA DO SERVIDOR**
> Os produtos guardados e o último endereço do checkout, novos na interface 1.2.0, ficam só no navegador, em `localStorage`: `cesto.guardados.<id do usuário>` e `cesto.endereco`. Não há tabela nem chave do Redis para eles.

Convenções

REGRA	EXEMPLO
Tabelas no plural, em snake_case	`order_items`
Colunas em snake_case; no código, camelCase via `@map`	`price_cents ↔ priceCents`
Chaves primárias UUID v4	`id UUID`
Dinheiro em centavos inteiros	`total_cents INTEGER`
Datas com fuso e milissegundos	`TIMESTAMPTZ(3)`
Enums nativos do PostgreSQL	`"OrderStatus"`
Restrições de integridade no banco	`CHECK (available >= 0)`
Índices nomeados como o Prisma	`orders_user_id_created_at_idx`

Nas tabelas de colunas a seguir, `PK` marca a chave primária e o ponto de interrogação no diagrama marca coluna que aceita nulo.

Copyright (©) 2026 Clara A. Portes · Licença MIT

2 cesto_auth

Usuários, clientes OAuth2, códigos de autorização, refresh tokens e as chaves que assinam os JWT.

Diagrama



users

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
id PK	UUID	não	uuid()	
email	VARCHAR(254)	não	sem padrão	Único (users_email_key), sempre minúsculo
name	VARCHAR(80)	não	sem padrão	
password_hash	TEXT	não	sem padrão	scrypt$N$sal$hash
role	"Role"	não	CUSTOMER	CUSTOMER ou ADMIN
created_at	TIMESTAMPTZ(3)	não	now()	
updated_at	TIMESTAMPTZ(3)	não	sem padrão	Atualizado pelo Prisma

Copyright (©) 2026 Clara A. Portes · Licença MIT

oauth_clients

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
id PK	VARCHAR(64)	não	sem padrão	Ex.: `cesto-web`, `cesto-internal`
name	VARCHAR(80)	não	sem padrão	
secret_hash	TEXT	sim	sem padrão	Nulo = cliente público (exige PKCE)
redirect_uris	TEXT[]	n/a	sem padrão	Endereços de retorno permitidos
grant_types	TEXT[]	n/a	sem padrão	authorization_code, refresh_token, client_credentials
scopes	TEXT[]	n/a	sem padrão	Teto de escopos do cliente
created_at	TIMESTAMPTZ(3)	não	now()	

authorization_codes

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
code_hash PK	CHAR(64)	não	sem padrão	SHA-256 do código
client_id	VARCHAR(64)	não	sem padrão	
user_id	UUID	não	sem padrão	FK users, ON DELETE CASCADE
redirect_uri	TEXT	não	sem padrão	Precisa bater na troca
code_challenge	VARCHAR(128)	não	sem padrão	Desafio PKCE S256
scope	TEXT	não	sem padrão	Escopos concedidos
expires_at	TIMESTAMPTZ(3)	não	sem padrão	Índice; 60 s por padrão
used_at	TIMESTAMPTZ(3)	sim	sem padrão	Uso único
created_at	TIMESTAMPTZ(3)	não	now()	

refresh_tokens

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
id PK	UUID	não	uuid()	
token_hash	CHAR(64)	não	sem padrão	Único; SHA-256 do token
family_id	UUID	não	sem padrão	Índice; une as rotações de um mesmo login
client_id	VARCHAR(64)	não	sem padrão	
user_id	UUID	não	sem padrão	FK users, ON DELETE CASCADE, índice
scope	TEXT	não	sem padrão	
expires_at	TIMESTAMPTZ(3)	não	sem padrão	30 dias por padrão

Copyright (©) 2026 Clara A. Portes · Licença MIT

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
`revoked_at`	`TIMESTAMPTZ(3)`	sim	sem padrão	Logout ou reuso detectado
`rotated_at`	`TIMESTAMPTZ(3)`	sim	sem padrão	Preenchido quando trocado por um novo
`created_at`	`TIMESTAMPTZ(3)`	não	`now()`	

signing_keys

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
`kid` PK	`VARCHAR(64)`	não	sem padrão	Vai no cabeçalho do JWT
`private_jwk`	`JSONB`	não	sem padrão	Chave privada RSA (proteja o backup)
`public_jwk`	`JSONB`	não	sem padrão	Publicada no JWKS
`active`	`BOOLEAN`	não	`true`	Chave usada para assinar
`created_at`	`TIMESTAMPTZ(3)`	não	`now()`	

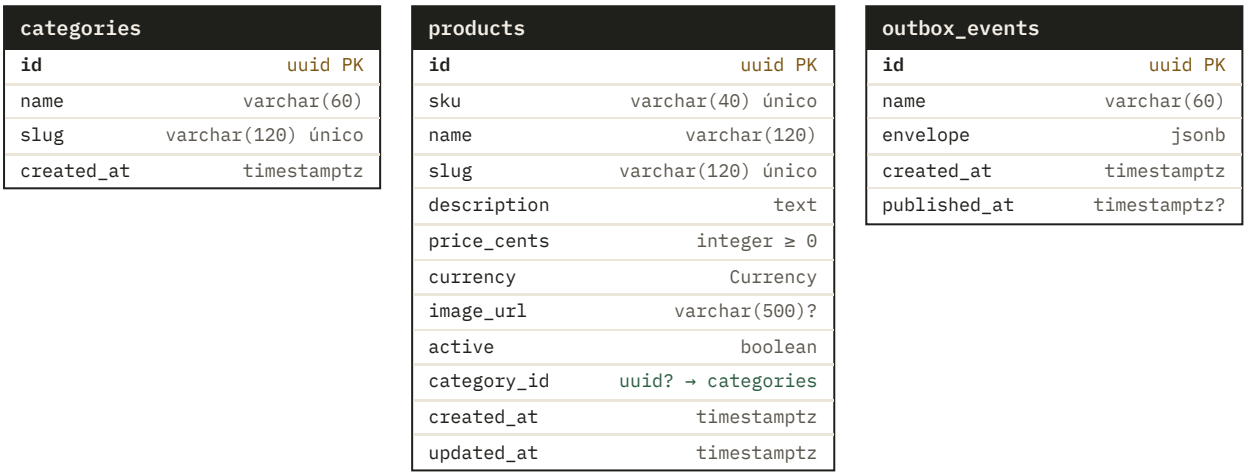
> **ATENÇÃO**
> A tabela `signing_keys` contém material criptográfico. Restrinja o acesso ao banco `cesto_auth` e criptografe os backups.

Copyright (©) 2026 Clara A. Portes · Licença MIT

3 cesto_catalog

Categorias e produtos da loja, mais o outbox que avisa os outros serviços das mudanças.

Diagrama



categories

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
id PK	UUID	não	uuid()	
name	VARCHAR(60)	não	sem padrão	
slug	VARCHAR(120)	não	sem padrão	Único; usado no filtro `?category=`
created_at	TIMESTAMPTZ(3)	não	now()	

products

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
id PK	UUID	não	uuid()	
sku	VARCHAR(40)	não	sem padrão	Único; maiúsculas, números e hífen
name	VARCHAR(120)	não	sem padrão	Busca sem diferenciar maiúsculas
slug	VARCHAR(120)	não	sem padrão	Único
description	TEXT	não	''	Busca
price_cents	INTEGER	não	sem padrão	`CHECK (price_cents >= 0)`

Copyright (©) 2026 Clara A. Portes · Licença MIT

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
currency	"Currency"	não	BRL	BRL, USD ou EUR
image_url	VARCHAR(500)	sim	sem padrão	
active	BOOLEAN	não	true	Desativar em vez de apagar
category_id	UUID	sim	sem padrão	FK categories, ON DELETE SET NULL, índice
created_at	TIMESTAMPTZ(3)	não	now()	
updated_at	TIMESTAMPTZ(3)	não	sem padrão	

O índice composto `(active, created_at)` atende a listagem padrão da loja, que mostra só produtos ativos, dos mais novos para os mais antigos.

outbox_events

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
id PK	UUID	não	sem padrão	Mesmo id do envelope do evento
name	VARCHAR(60)	não	sem padrão	Nome do evento
envelope	JSONB	não	sem padrão	Envelope completo, validado com Zod
created_at	TIMESTAMPTZ(3)	não	now()	
published_at	TIMESTAMPTZ(3)	sim	sem padrão	Preenchido pelo relay

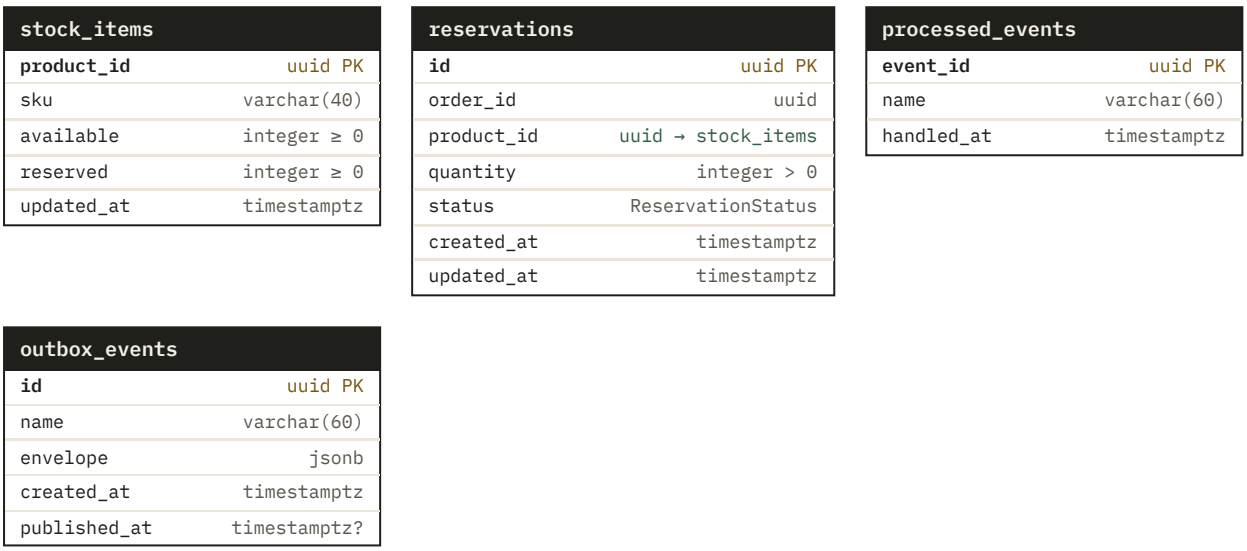
O índice `(published_at, created_at)` deixa o relay achar rápido o que falta publicar, na ordem. A mesma tabela existe em `cesto_inventory`, `cesto_orders` e `cesto_payments`.

Copyright (©) 2026 Clara A. Portes · Licença MIT

4 cesto_inventory

O saldo de cada produto e as reservas presas aos pedidos em andamento.

Diagrama



stock_items

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
`product_id` PK	`UUID`	não	sem padrão	Mesmo id do catálogo
`sku`	`VARCHAR(40)`	não	sem padrão	Cópia para leitura
`available`	`INTEGER`	não	`0`	`CHECK (available >= 0)`; pode ser vendido
`reserved`	`INTEGER`	não	`0`	`CHECK (reserved >= 0)`; preso a pedidos em andamento
`updated_at`	`TIMESTAMPTZ(3)`	não	sem padrão	

reservations

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
`id` PK	`UUID`	não	`uuid()`	
`order_id`	`UUID`	não	sem padrão	Único com `product_id`
`product_id`	`UUID`	não	sem padrão	FK stock_items, ON DELETE RESTRICT
`quantity`	`INTEGER`	não	sem padrão	`CHECK (quantity > 0)`

Copyright (©) 2026 Clara A. Portes · Licença MIT

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
status	"ReservationStatus"	não	HELD	HELD, COMMITTED ou RELEASED
created_at	TIMESTAMPTZ(3)	não	now()	Índice (status, created_at)
updated_at	TIMESTAMPTZ(3)	não	sem padrão	

processed_events e outbox_events

processed_events registra cada evento já consumido, para que uma entrega repetida seja ignorada. Existe também em cesto_orders e cesto_payments.

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
event_id PK	UUID	não	sem padrão	Id do evento consumido
name	VARCHAR(60)	não	sem padrão	
handled_at	TIMESTAMPTZ(3)	não	now()	

outbox_events tem a mesma estrutura descrita em cesto_catalog:

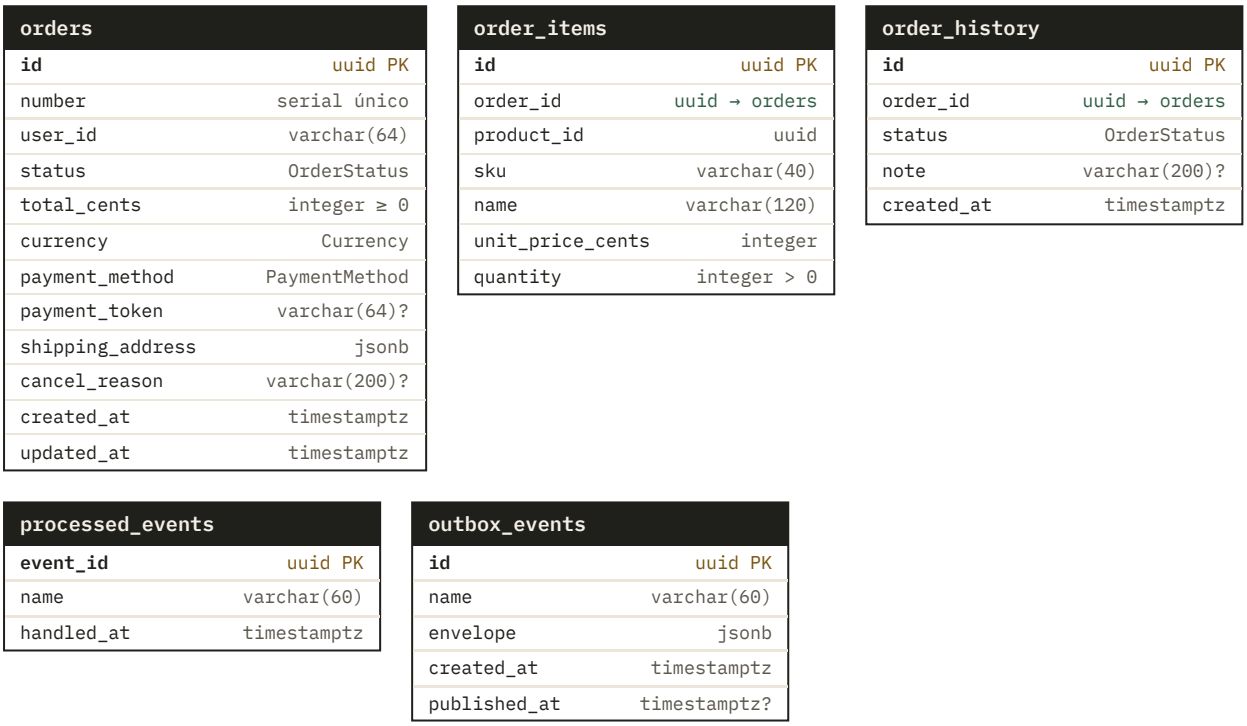
COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
id PK	UUID	não	sem padrão	Mesmo id do envelope do evento
name	VARCHAR(60)	não	sem padrão	Nome do evento
envelope	JSONB	não	sem padrão	Envelope completo, validado com Zod
created_at	TIMESTAMPTZ(3)	não	now()	
published_at	TIMESTAMPTZ(3)	sim	sem padrão	Preenchido pelo relay

Copyright (©) 2026 Clara A. Portes · Licença MIT

5 cesto_orders

Pedidos, os itens com preço congelado e o histórico de cada mudança de status.

Diagrama



orders

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
id PK	UUID	não	uuid()	
number	SERIAL	não	sequência	Único; número amigável mostrado na loja (#1021, #1022…)
user_id	VARCHAR(64)	não	sem padrão	sub do JWT; índice (user_id, created_at)
status	"OrderStatus"	não	PENDING	Índice (status, created_at)
total_cents	INTEGER	não	sem padrão	CHECK (total_cents >= 0)
currency	"Currency"	não	BRL	
payment_method	"PaymentMethod"	não	sem padrão	card, pix ou boleto

Copyright (©) 2026 Clara A. Portes · Licença MIT

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
payment_token	VARCHAR(64)	sim	sem padrão	Token do provedor (nunca o cartão)
shipping_address	JSONB	não	sem padrão	Endereço validado pelo esquema `Address`
cancel_reason	VARCHAR(200)	sim	sem padrão	
created_at	TIMESTAMPTZ(3)	não	now()	
updated_at	TIMESTAMPTZ(3)	não	sem padrão	

order_items

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
id PK	UUID	não	uuid()	
order_id	UUID	não	sem padrão	FK orders, ON DELETE CASCADE, índice
product_id	UUID	não	sem padrão	Referência lógica ao catálogo
sku / name	VARCHAR(40) / VARCHAR(120)	não	sem padrão	Cópia no momento da compra
unit_price_cents	INTEGER	não	sem padrão	Preço congelado
quantity	INTEGER	não	sem padrão	`CHECK (quantity > 0)`

order_history

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
id PK	UUID	não	uuid()	
order_id	UUID	não	sem padrão	FK orders, ON DELETE CASCADE; índice `(order_id, created_at)`
status	"OrderStatus"	não	sem padrão	Status atingido
note	VARCHAR(200)	sim	sem padrão	Motivo ou observação
created_at	TIMESTAMPTZ(3)	não	now()	

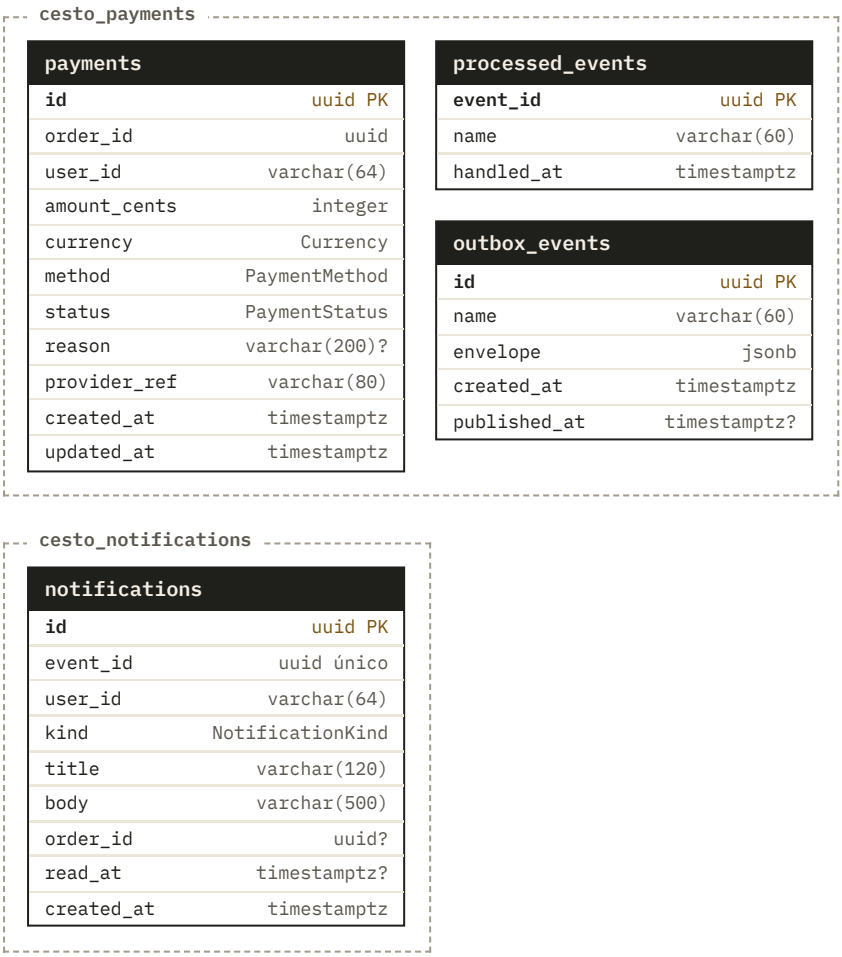
`processed_events` e `outbox_events` seguem a estrutura descrita nos capítulos 3 e 4. A trilha da saga que a tela Pedidos mostra vem de `order_history`.

Copyright (©) 2026 Clara A. Portes · Licença MIT

6 cesto_payments e cesto_notifications

As cobranças e estornos de cada pedido, e as notificações geradas a partir dos eventos.

Diagrama



As linhas tracejadas separam os dois bancos. `processed_events` e `outbox_events` pertencem a `cesto_payments`; em `cesto_notifications`, a coluna única `event_id` faz o papel da deduplicação.

payments

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
`id` PK	UUID	não	`uuid()`	
`order_id`	UUID	não	sem padrão	Índice

Copyright (©) 2026 Clara A. Portes · Licença MIT

COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
user_id	VARCHAR(64)	não	sem padrão	Índice (user_id, created_at)
amount_cents	INTEGER	não	sem padrão	
currency	"Currency"	não	BRL	
method	"PaymentMethod"	não	sem padrão	
status	"PaymentStatus"	não	sem padrão	APPROVED, DECLINED ou REFUNDED
reason	VARCHAR(200)	sim	sem padrão	Motivo da recusa ou do estorno
provider_ref	VARCHAR(80)	não	sem padrão	Referência no provedor
created_at	TIMESTAMPTZ(3)	não	now()	
updated_at	TIMESTAMPTZ(3)	não	sem padrão	

notifications

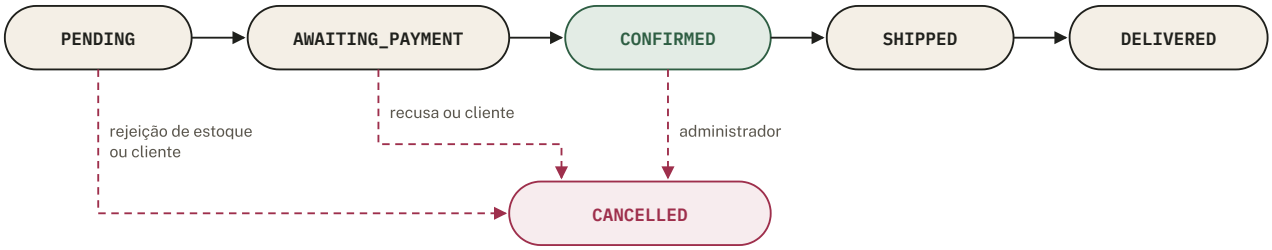
COLUNA	TIPO	NULO	PADRÃO	OBSERVAÇÃO
id PK	UUID	não	uuid()	
event_id	UUID	não	sem padrão	Único; impede notificação duplicada
user_id	VARCHAR(64)	não	sem padrão	Índices (user_id, created_at) e (user_id, read_at)
kind	"NotificationKind"	não	sem padrão	WELCOME, ORDER_PLACED, ORDER_CONFIRMED, ORDER_CANCELLED, ORDER_SHIPPED, ORDER_DELIVERED, PAYMENT_DECLINED
title	VARCHAR(120)	não	sem padrão	
body	VARCHAR(500)	não	sem padrão	
order_id	UUID	sim	sem padrão	
read_at	TIMESTAMPTZ(3)	sim	sem padrão	Nulo = não lida
created_at	TIMESTAMPTZ(3)	não	now()	

Copyright (©) 2026 Clara A. Portes · Licença MIT

7 Máquinas de estado

Os status que um pedido, uma reserva, um refresh token e um pagamento podem ter, e o que os faz mudar.

Pedido



DE	PARA	GATILHO
PENDING	AWAITING_PAYMENT	`inventory.reserved`
PENDING	CANCELLED	`inventory.rejected` ou cliente
AWAITING_PAYMENT	CONFIRMED	`payment.approved`
AWAITING_PAYMENT	CANCELLED	`payment.declined` ou cliente
CONFIRMED	SHIPPED	Administrador
CONFIRMED	CANCELLED	Administrador
SHIPPED	DELIVERED	Administrador

A troca de status usa `UPDATE ... WHERE id = ? AND status = ?`. Se duas mensagens tentarem mudar o mesmo pedido ao mesmo tempo, só uma vence, e cada mudança gera uma linha em `order_history`.

Reserva de estoque

STATUS	SIGNIFICADO	EFEITO NO SALDO
HELD	Pedido em andamento	`available -= q, reserved += q`
COMMITTED	Pedido confirmado	`reserved -= q` (baixa definitiva)
RELEASED	Pedido cancelado	Se estava HELD: `reserved -= q, available += q`. Se estava COMMITTED: `available += q`

Copyright (©) 2026 Clara A. Portes · Licença MIT

Refresh token

SITUAÇÃO	COLUNAS	RESULTADO
Ativo	`rotated_at` e `revoked_at` nulos	Pode ser trocado uma vez
Rotacionado	`rotated_at` preenchido	Reuso revoga toda a família
Revogado	`revoked_at` preenchido	Logout, reuso ou código reaproveitado
Expirado	`expires_at` no passado	Apagado pela limpeza horária

Pagamento

STATUS	QUANDO
`APPROVED`	Cobrança aceita pelo provedor
`DECLINED`	Cobrança recusada; o motivo fica em `reason`
`REFUNDED`	Pedido cancelado depois da aprovação

Copyright (©) 2026 Clara A. Portes · Licença MIT

8 Eventos, outbox e Redis

Como um evento sai do banco, chega ao Redis e é lido pelos outros serviços.

Outbox transacional

1. A mudança de negócio e a linha em `outbox_events` são gravadas na mesma transação.
2. O relay de cada serviço lê até 100 linhas com `published_at IS NULL`, em ordem de criação, publica com `XADD` e marca `published_at`.
3. Se o processo cair entre publicar e marcar, a linha é publicada de novo; quem consome descarta a repetição pela tabela `processed_events`.

```
SELECT name, count(*) FROM outbox_events WHERE published_at IS NULL GROUP BY name;
DELETE FROM outbox_events WHERE published_at < now() - interval '30 days';
```

Chaves do Redis

CHAVE	TIPO	DONO	CONTEÚDO
`cesto:events:<evento>`	Stream	quem publica	Envelope JSON no campo `event`; limite aproximado de 100 mil entradas
`cesto:events:dead-letter`	Stream	todos	Eventos inválidos ou que falharam 5 vezes, com stream, id, grupo e motivo
`cesto:cart:<userId>`	Hash	`cart`	productId → quantidade; expira em 30 dias sem uso
`cesto:ratelimit:*`	String	`gateway`	Contadores por IP

Cada serviço consumidor tem um grupo com o próprio nome (`inventory`, `orders`, `payments`, `notifications`, `cart`) em cada stream que lê. A página Serviços da interface mostra se o Redis está respondendo, pelo `GET /health` do gateway.

Envelope de evento

```
{
  "id": "b0c5...", "name": "order.placed", "version": 1,
  "occurredAt": "2026-10-22T16:57:02.311Z", "correlationId": "req-...",
  "payload": { "orderId": "...", "userId": "...", "number": 1021, "items": [ ... ],
               "totalCents": 36470, "currency": "BRL", "paymentMethod": "pix" }
}
```

O envelope e o payload são validados com os esquemas Zod de `packages/contracts/src/events.js` ao publicar e de novo ao ler.

Copyright (©) 2026 Clara A. Portes · Licença MIT

9 Como preparar o banco

Com Docker, num PostgreSQL próprio, ao mudar o esquema e na hora do backup.

Com Docker (recomendado)

O `docker-compose.yml` sobe o PostgreSQL 16 e executa `docker/postgres/init.sql` na primeira vez, criando os seis bancos. Cada serviço roda `prisma migrate deploy` ao iniciar, com até 10 tentativas enquanto o banco sobe, e depois executa `node src/main.js`. Não há passo manual.

PostgreSQL próprio

```
CREATE ROLE cesto LOGIN PASSWORD 'troque-esta-senha';
CREATE DATABASE cesto_auth OWNER cesto;
CREATE DATABASE cesto_catalog OWNER cesto;
CREATE DATABASE cesto_inventory OWNER cesto;
CREATE DATABASE cesto_orders OWNER cesto;
CREATE DATABASE cesto_payments OWNER cesto;
CREATE DATABASE cesto_notifications OWNER cesto;
```

Depois, com `DB_USER`, `DB_PASSWORD`, `DB_HOST` e `DB_PORT` no `.env`:

```
npm ci
npm run generate   # gera o cliente Prisma de cada serviço em src/generated/prisma
npm run migrate
```

Em servidores separados, defina o `DATABASE_URL` de cada serviço diretamente no ambiente.

Mudar o esquema

1. Edite `services/<serviço>/prisma/schema.prisma`.
2. Na pasta do serviço, rode `npx prisma migrate dev --name descricao` apontando para um banco de desenvolvimento.
3. Revise o SQL gerado em `prisma/migrations/` e acrescente restrições `CHECK` quando fizer sentido.
4. Rode `npm run generate` para atualizar o cliente Prisma e `npm test`. Como não há compilador, procure no código os usos das colunas alteradas (`grep` pelo nome em camelCase) e deixe os testes apontarem o resto.
5. Em produção, `prisma migrate deploy` aplica as migrações novas ao iniciar o serviço.

Copyright (©) 2026 Clara A. Portes · Licença MIT

Backup e retenção

ITEM	RECOMENDAÇÃO
Bancos	`pg_dump -Fc` diário de cada banco; teste a restauração
`cesto_auth`	Backup criptografado (contém chaves de assinatura e hashes)
`outbox_events / processed_events`	Apague registros com mais de 30 dias
`notifications`	Mantenha conforme a política da loja
Redis	AOF ligado no compose; o carrinho é o único dado de negócio

```
for db in auth catalog inventory orders payments notifications; do
  docker compose exec -T postgres pg_dump -U cesto -Fc cesto_$db > backup-$db.dump
done
```

Consultas úteis

Pedidos e valor total por status:

```
SELECT status, count(*), sum(total_cents) / 100.0 AS total FROM orders GROUP BY status;
```

Linha do tempo de um pedido:

```
SELECT o.number, h.status, h.note, h.created_at
FROM order_history h JOIN orders o ON o.id = h.order_id
WHERE o.number = 1021 ORDER BY h.created_at;
```

Produtos com pouco estoque:

```
SELECT sku, available, reserved FROM stock_items WHERE available < 5 ORDER BY available;
```

Reservas presas há mais de 15 minutos:

```
SELECT r.order_id, r.status, s.sku, r.quantity
FROM reservations r JOIN stock_items s USING (product_id)
WHERE r.status = 'HELD' AND r.created_at < now() - interval '15 minutes';
```

Copyright (©) 2026 Clara A. Portes · Licença MIT